AWSKRUG Frontend Meetup

Can You Trust E2E Tests That Pass?

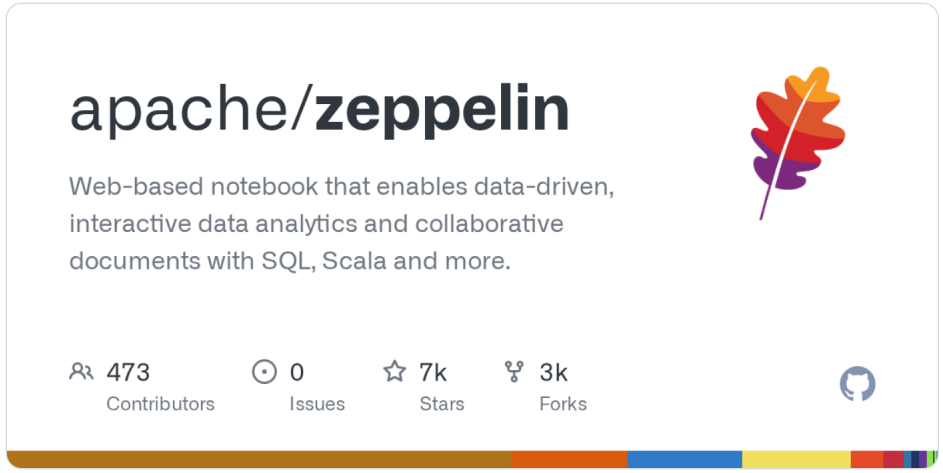
Yongjae Lee - Web FE Developer

LinkedIn/GitHub: **@voidmatcha**

SNOW

The Problem

Joined 2025 OSSCA as a Zeppelin mentee



Built E2E setup from scratch

Wrote the tests with an LLM

(with Claude Opus 4.1)

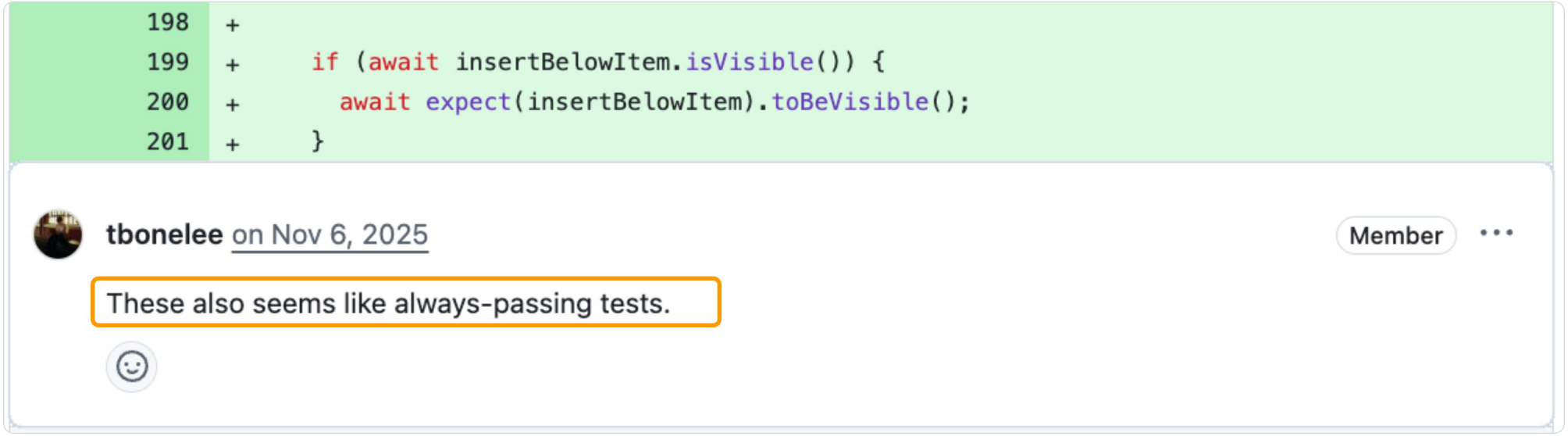
More LLM errors than expected

Often violates principles like YAGNI and DRY

Tests that pass but verify nothing

Reviews I received

Learning "always-passing tests" from PR reviews



Actually merged code

When does this test fail?

calcom/cal.diy / PR #28486 (merged)

```
await expect(page.getByTestId("away-emoji")).toBeTruthy();
```

Does it fail if away-emoji is not on screen?

Answer: No. A Locator object always exists

Why does it pass?

Passes even without the element

```
$ npx playwright test

✓  1 away emoji - before (toBeTruthy) (83ms)
✘  2 away emoji - after (toBeVisible) (2.1s)
   ⋮
   Error: element(s) not found
   ⋮
  1 failed
  1 passed (3.1s)
```

A Locator is a "way to find" the element, so the object always exists

Similar cases

`if (isVisible)`	`.catch(() ⇒ {})`	`test.only`	**Optimistic UI only**

Playwright Locators / Official docs

Others hit similar problems

Kent Beck: agents delete assertions and tests



Kent Beck

Creator of XP
"Test-Driven Development: By Example" author

Now the complexity has gotten to be too much for the genie. There's no path consisting only of tiny steps getting from here to there.

The genie doesn't respond well to this situation. I've seen infinite loops. I've seen truly pernicious behavior like deleting assertions from tests, deleting whole tests, & faking large swathes of implementation. These behaviors may allow the genie to reward itself with a biscuit, but they destroy trust, my trust. And just when I was starting to have fun programming again...

When the idea of LLM skills was just emerging

So I started building it as skills

voidmatcha/**e2e-skills**



Playwright and Cypress skills for Claude Code and Codex: generate tests, catch false-green tests and test smells, debug failures.

3 Contributors · 2 Issues · 21 Stars · 3 Forks

github.com/voidmatcha/e2e-skills

First step of e2e-skills

Start with best practices research

Official docs

Playwright / Cypress guides

Failures I hit myself

Issues reproduced in Zeppelin and elsewhere

Playwright Best Practices / Cypress Best Practices

Step 1 result / 24 rules by severity

The 24 rules this produced

P0 / Always passes

Passes when the product breaks

#1 Name/assertion mismatch
#2 No final-state check
#3 Swallowed errors (+#3b)
#4 Meaningless assertion
#5 Bypass conditional
#7 Leftover test.only
#8 Missing assertion
#12 Missing auth setup

P1 / Hard to diagnose

Unclear what is wrong when it fails

#6 Raw DOM query
#9 Fixed sleep
#10 Flaky pattern
#13 POM mismatch
#14 Hard-coded account
#15 Missing expect await
#16 Missing action await
#17 Direct page selector
#18 expect.soft leak
#19 Module-level state
#20 Writes to real server
#22 Unchecked optimistic UI

P2 / Upkeep

Costs pile up

#11 YAGNI · zombie spec
#21 Relies on manual session file
#23 Fixture ignores render conditions

From LLM-only to lint + LLM

LLM-only produces many false positives

0.39

Precision (LLM only, no rules)

27

Stable false positives across repeated runs

Switched to lint + LLM: lint filters candidates, the LLM judges

36 cases × 5 runs / gpt-6-sol / development stage (public cases)

As I applied lint…

Added the rule I needed to eslint-plugin-playwright

mskelton/eslint-plugin-playwright #470

`no-unnecessary-assertions`

Replace assertions that pass even when nothing is on screen with toBeVisible()

2026-07-21 merged / v2.11 recommended

Next after lint / AST-based scanner

The scanner picks candidates; LLM review confirms

```
$ scan.sh tests/
[AST][LLM-TRIAGE] sg-4f-locator-as-truthy (1 hit;
Playwright expect provenance unproven)
  …/tests/dashboard.spec.ts:5:9
```

Tier 1	Tier 2	Tier 3
ESLint	ast-grep	rg + Python

Verifying that it can actually fail

Not "Did it pass?" but "Can it fail?"

scanner
Candidate selection / reviewer

→

LLM review
Refute false positives

→

Execution check
e2e-reviewer opt-in /
V2·V3 probe

Check failure first

Willison: a test that has never failed can't be trusted



Simon Willison

Datasette
Agentic Engineering Patterns
Guide author

It's important to confirm that the tests fail before implementing the code to make them pass. If you skip that step you risk building a test that passes already, hence failing to exercise and confirm your new implementation.

That's what "red/green" means: the red phase watches the tests fail, then the green phase confirms that they now pass.

Checking by injecting faults (e2e-reviewer opt-in)

Negate assertions, inject faults, confirm failure

V2: assertion negate

```
// original
await expect(status).toHaveText("Count: 1");
// negated copy
await expect(status).not.toHaveText("Count: 1");
```

V3: inject a product fault

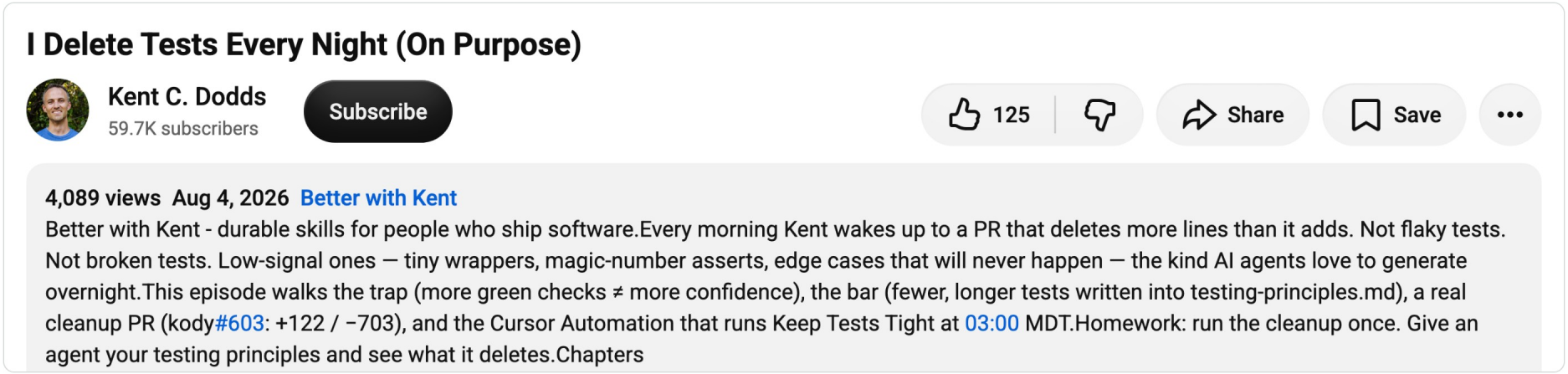
```
// fault: count response set to 0
await page.route("**/count", r ⇒
  r.fulfill({ json: { count: 0 } }));
// assertion unchanged: must fail here
await expect(status).toHaveText("Count: 1");
```

The question again

They say models are better now, is this still needed?

Similar claims are still appearing

Dodds: agents write unreliable tests every night



Similar claims are still appearing

Thoughtworks: mutation testing for always-pass tests

/thoughtworks

Technology Radar
Technique / Trial ring
2026-04-15

With AI-generated test cases now commonplace, mutation testing acts as a reinforcement layer for catching "perpetually green" tests — those that pass regardless of logic changes due to missing assertions or decoupled mocks. By using tools such as Stryker, Pitest or cargo-mutants, we shift the focus from how much code is executed to how much code is actually verified, particularly in core domain logic. The goal is to ensure that a passing test suite is a reliable signal of functional correctness, rather than simply a report of which lines were executed.

More evidence (1/3) / newest first

Practitioner posts and talks

Source	Key point
Paul Stack (Swamp) / 2026-09-23	Tests an agent writes are specs, not verification
Michael Bromley / 2026-09-07	Classifies tautological tests, weak matchers, conditional assertions
Toss FE webinar / 2026-09-06	Session on how to verify AI-made output
Tariq Shaukat (Sonar) / 2026-07-21	Talk: the bottleneck moved from generation to verification
Jeff Thoensen, DEV / 2026-07-14	Reset test checked only an on-screen message / real bug found
NAVER D2 / 2026-06-15	Session on trusting agent-written code via a Playwright E2E harness
Tian Pan / 2026-05-17	Tests written from the code pass instantly / assertion laundering
Dave Farley, Aviator / 2026-02-13	Why AI makes verification the bottleneck

More evidence (2/3) / newest first

Related measurements and cases

Source	Key point
Ivan Davidov, YouTube / 2026-09-09	4 Claude Code runs, same prompt; one had a false-positive assertion
Malhotra, SD Times / 2026-09-02	After an AI fix, a test can go green while checking the wrong element
Szymon Fiedler, RBQ 2026 / 2026-08-11	Mutation testing talk asking whether passing tests really verify
Paul Newell, DEV / 2026-06-30	All 20 AI-written tests passed, but 27 of 95 mutants survived
Lee, arXiv / 2026-05-02	Auto test repair converges by weakening assertions or deleting tests
Gleb Bahmutov (Mercari US) / 2026-04-28	AI-made assertion only checks that the body is visible
Senko Rašić / 2026-03-22	98% coverage, yet only 30 of 40 LLM-proposed mutants caught (75%)
David R. MacIver, Antithesis / 2025-12-10	LLM unit tests are weak guardrails; use property-based tests

More evidence (3/3) / newest first

Research, official docs, survey

Source	Key point
Vitest guide / captured 2026-10-12	Warns that tests which only call and check for a throw give false confidence
Currents / 2026-08-27	Healer widens the locator so the test can no longer fail
Kaufman et al., arXiv / 2026-07-09	Programmers spotted wrong assertions only 49% of the time (n=86)
Teoh et al., WebTestPilot / 2026-02-12	96% precision and recall on 100 injected bugs across 4 web apps
Sonar survey / 2026-01-08	96% don't fully trust AI code; only 48% always check before commit
Mathews et al., arXiv / 2024-12-18	Paper: LLM test generator design choices block bug discovery
Alshahwan et al., Meta / 2024-02-14	Only 25% of generated tests raised coverage; fault detection not shown
Anthropic, Best practices for Claude Code	Without a runnable check, 'it looks done' is the only signal

Conclusion

Problems remain even as models improve

PASS ≠ verification

A Locator always exists

Healer ambiguity

Test fault vs product fault unclear

Verification bottleneck

Checking is harder than generating

"PASS" must mean "it works" to be trusted

Back to the tool

Introducing voidmatcha/e2e-skills

`playwright-test-generator`

`e2e-reviewer`

`playwright-debugger`

`cypress-debugger`

github.com/voidmatcha/e2e-skills / skills

e2e-reviewer - benchmark

False positives down from 27 to 2

Full skill

0.95

Precision

2 false positives

100% of defects found

Pattern list only

0.89

Precision

5 false positives

100% of defects found

No rules

0.39

Precision

27 false positives

43% of defects found

F1 difference	Criterion	Result (95% CI)	Verdict
Full vs no rules	≥ +0.3, lower bound > 0	+0.571 (+0.411 ~ +0.738)	**Pass**
Full vs list only	No criterion set	+0.034 (0.000 ~ +0.072)	Reference

36 cases × 5 runs / gpt-6-sol / development stage (public cases)

Field test in progress / merged

15 open-source PRs merged

```
203   -   await expect(page.getByTestId("away-emoji")).toBeTruthy();
203   +   await expect(page.getByTestId("away-emoji").first()).toBeVisible();
```

Cal.com #28486 2026-03-25 merged


★ 91.2k


★ 79.6k


★ 73.3k



★ 55.5k

★ 49.0k


★ 47.5k


★ 22.1k


★ 20.8k


★ 13.6k


★ 11.2k


★ 9.5k


★ 7.4k

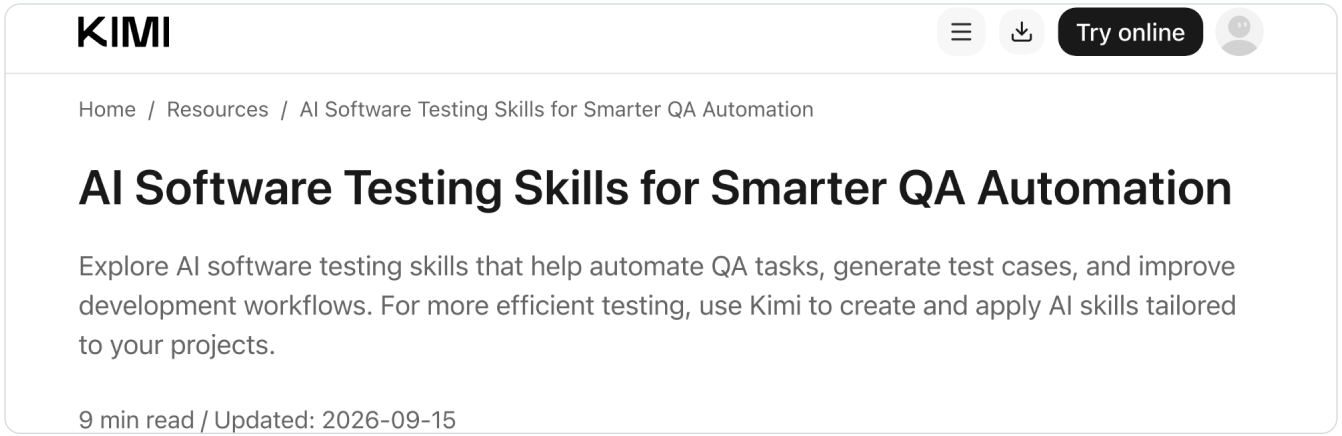

★ 6.7k


★ 5.9k


★ 2.7k

External mention

Listed on Kimi's official testing-skills page



e2e-skills	AI agent testing toolkit for Playwright and Cypress: generate E2E tests from scratch, review existing specs against 24 anti-patterns (P0/P1/P2 silent-always-pass smells), and debug flaky failures from playwright-report/ or cypress/reports/.	https://github.com/voidmatcha/e2e-skills

kimi.ai/resources/software-testing-skills

v3.0.0

Improved while preparing this talk

generator	6.74x slower / fault detection unproven	→	**Delegate to the official agents; add only rules and the scanner**
debugger	7-10x input / no gain	→	**Slimmed skill / 0.69x input / similar accuracy**

v3.0.0

Current generator structure and flow



Playwright Test Agents / Playwright 1.56 or later

Roadmap

What I want to improve next

Two areas not covered yet

Retries and skips

Problem Failures hidden by retries and skip

Approach Scanner finds them in config/code

Mock vs real API drift

Problem Mocks pass; the real API breaks

Approach LLM compares mock vs API spec

Not yet measured: state and auth

A question to take home

Have you felt safe about tests an LLM wrote?

Thank you